DEEP LEARNING SCHOOL

"ADVANCED CONCEPTS"

4TH EDITION

Normalizing Flows

Deep Learning School "Advanced Concepts”

José I. Robledo - 2026

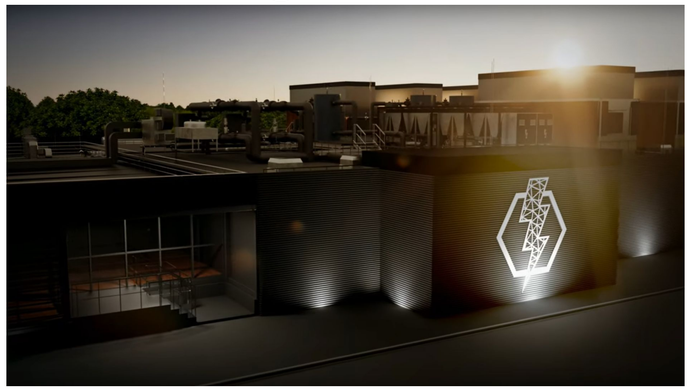
ERUM DATA HUB

Introduction

Who am I?

Lecture and tutorial: José Ignacio Robledo

- Researcher at the National Scientific Council of Argentina (CONICET): Neutrons and AI
- Professor at the Balseiro Institute, Argentina. Physics based deep learning course.
- AI Consultant at the Jülich Supercomputing Centre of Forschungszentrum Jülich

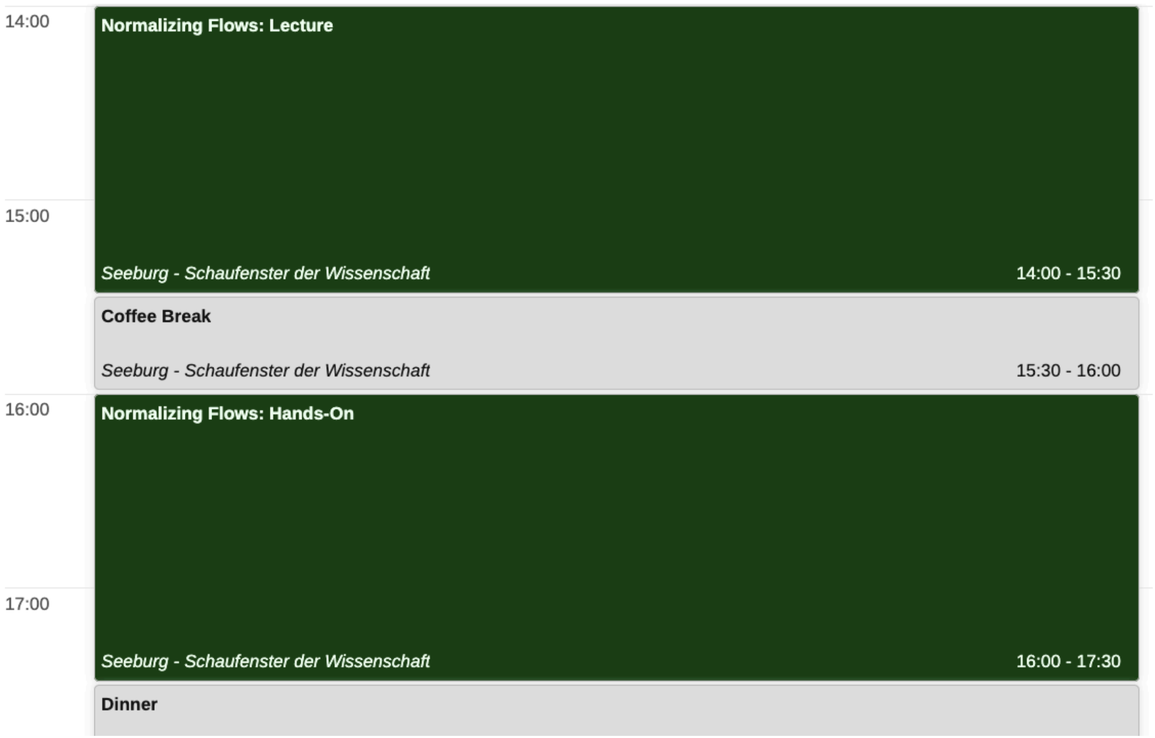


Introduction

Schedule Wednesday 02 September

What you saw today

What we will see now!

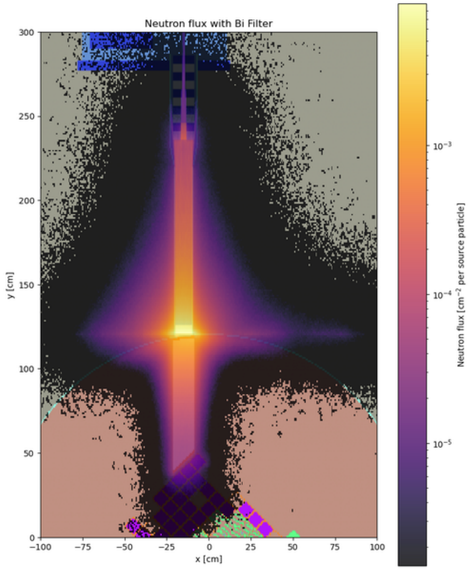


ERUM DATA HUB

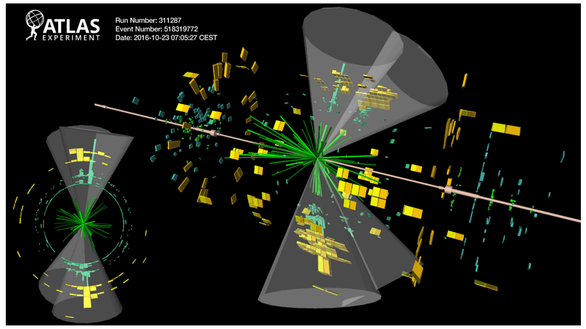
Introduction

Physical systems generate complex distributions.

Many times we observe samples from a system or simulator, but we do not have a tractable expression for their probability density



$$\{x_i\}_{i=1}^{N} \sim p_{physics}(x)$$ *Unknown!*



Nicole Hartman (2023)

Deterministic Model

$$x \to \hat{y}$$

Probabilistic model

$$x \to \hat{p}(y \mid x)$$

Simple parametric distributions are tractable but not expressive enough!

Can we build a model that is both flexible and probabilistically tractable?

ERUM DATA HUB

Probabilistic models

Distributions and uncertainty

- We’re not interested in modeling a specific value, rather in learning the distribution that generated it!
- Learning a **probability distribution** allows us to *model uncertainty* inherently existent in natural systems
- Several sources of variability: noise, unknown conditions, non-observed interactions, or even the stochastic nature of a system.
- **A probabilistic model defines a distribution over the data in an implicit or explicit manner, which can be used to generate new samples of data.**

The learning of a probability distribution from the data allows to capture patterns, correlations and structures that are underlying and maybe non-evident with deterministic models

Learning a distribution from the data

Loss function

- Given data $D = \{x_i\}_{i=1,\dots,n}$ we search for a model with parameters θ such that it learns the true distribution of the data p,

$$p(D) \text{ o } p(x)$$

- We search for a model that *generates* data with certain probability. If p_θ is our parametric model (θ the parameters of our generative model) then we search for:

Maximum log-likelihood Estimation

$$\theta^* = \arg\max_\theta \log p_\theta(D) \overset{i.i.d}{=} \arg\max_\theta \frac{1}{n} \sum_{i=1}^{n} \log p_\theta(x_i)$$

$$\implies \theta^* \approx \arg\max_\theta \mathbb{E}_{X \sim p(x)} \left[\log p_\theta(X)\right]$$ **Expected maximum log-likelihood**

ERUM DATA HUB

Learning a distribution from the data

Simple models

If $x \in \mathbb{R}^d$, we could model $p_\theta(x)$ with a known parametrized analytical distribution, e.g.

$$x \sim p_\theta(x) = \mathcal{N}(x \,|\, \mu, \Sigma), \text{ con } \theta = \{\mu, \Sigma\}$$

We can fit the multivariate gaussian to the data by Maximum Likelihood Estimation (MLE), no problem.

- **Unimodal** model
- Impossible to capture all features from a complex distribution
- *We are in need of more expressive models*

https://en.wikipedia.org/wiki/Spherical_cow

Learning a distribution from the data

Kullback-Leibler Divergence

- The **Kullback-Leibler Divergence** (KL) between P and Q is the relative entropy between both distributions. It is a *measure of information* that quantifies the similarity of probability distribution $P(X)$ with respect to that of the candidate $Q(X)$.
- We know that the entropy is a measure of average information (or uncertainty) of a random variable X. It is defined as (Shannon, 1948)

$$\mathbb{H}(P) = -\sum_{x \in X} P(x) \log P(x) = -\mathbb{E}_{x \sim P}[\log P(x)],$$

Where X is sampled from the distribution P.

Learning a distribution from the data

Kullback-Leibler Divergence

- We define the KL-divergence as

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)} = \underbrace{-\mathbb{H}(P)}_{\textit{Entropy}} + \underbrace{\mathbb{H}(P,Q)}_{\textit{Cross Entropy!}}$$

It can be interpreted as the average additional information required to code the data using the candidate probability distribution instead of the original one.

Continuous version: $D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_{x \sim P} \left[\log \frac{p(x)}{q(x)} \right]$

Learning a distribution from the data

From MLE to KL-divergence

- It turns out that **maximizing the expected log-likelihood is equivalent to minimizing the KL-divergence** between the real distribution and that of the parametric model:

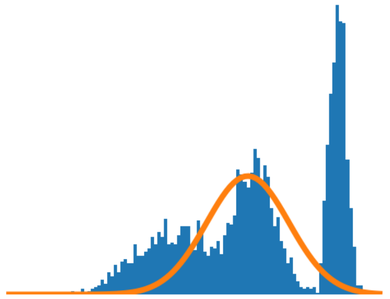
$$\text{Since } D_{KL}(p(x)\,|\,p_\theta(x)) = \mathbb{E}_{x\sim p(x)}\left[\log \frac{p(x)}{p_\theta(x)}\right], \textbf{ then}$$

$$\theta^* = \arg\min_\theta D_{KL}\big(p(x)\,|\,p_\theta(x)\big) = \arg\max_\theta \mathbb{E}_{x\sim p(x)}\left[\log p_\theta(x)\right]$$

For normalizing flows, this is exactly why maximum-likelihood training encourages the learned density $p_\theta(x)$ to cover the regions occupied by the data.

Normalizing Flows

Introduction to NFs



- We can state a model that attempts to learn directly $p(x)$.
- Let's model a complex target distribution $p(x)$, with $x \in \mathbb{R}^D$, **transforming a simple and well known distribution by means of a set of invertible functions until we reach** $p(x)$.
- Start from a *d* dimensional Gaussian

$$z \sim p_z(z) = \mathcal{N}_D(z \,|\, \mu, \sigma^2)$$

Now, start cooking! Shaping the dough → Transformations

Central question: **How is the distribution of a random variable transformed under a change of variable?**

Normalizing Flows

Transformation of a distribution under a change of variables

- We will **demand** for an *invertible* transformation $f : \mathbb{R}^D \to \mathbb{R}^D$. Let's denote its inverse as f^{-1}.

- Let $\vec{z} \in \mathbb{R}^D$. When we apply the transformation, we obtain a new variable $y = f(\vec{z}) \in \mathbb{R}^D$. How is now the distribution of y when compared to that of z? i.e., how has the probability density of the new random variable Y changed with respect to that of Z?

$$\int p_Y(\vec{y}) d\vec{y} = \int p_Z(\vec{z}) d\vec{z} = 1 \quad \Longrightarrow \quad p_Y(\vec{y}) = p_Z(f^{-1}(\vec{y})) \left| \det \frac{\partial f^{-1}(\vec{y})}{\partial \vec{y}} \right|$$

Jacobian

Normalizing Flows

Transformation of a distribution under a change of variables

Example $D = 1$

$$z \sim U[0,1] \qquad p_Z(z) = 1, \forall z \in [0,1]$$

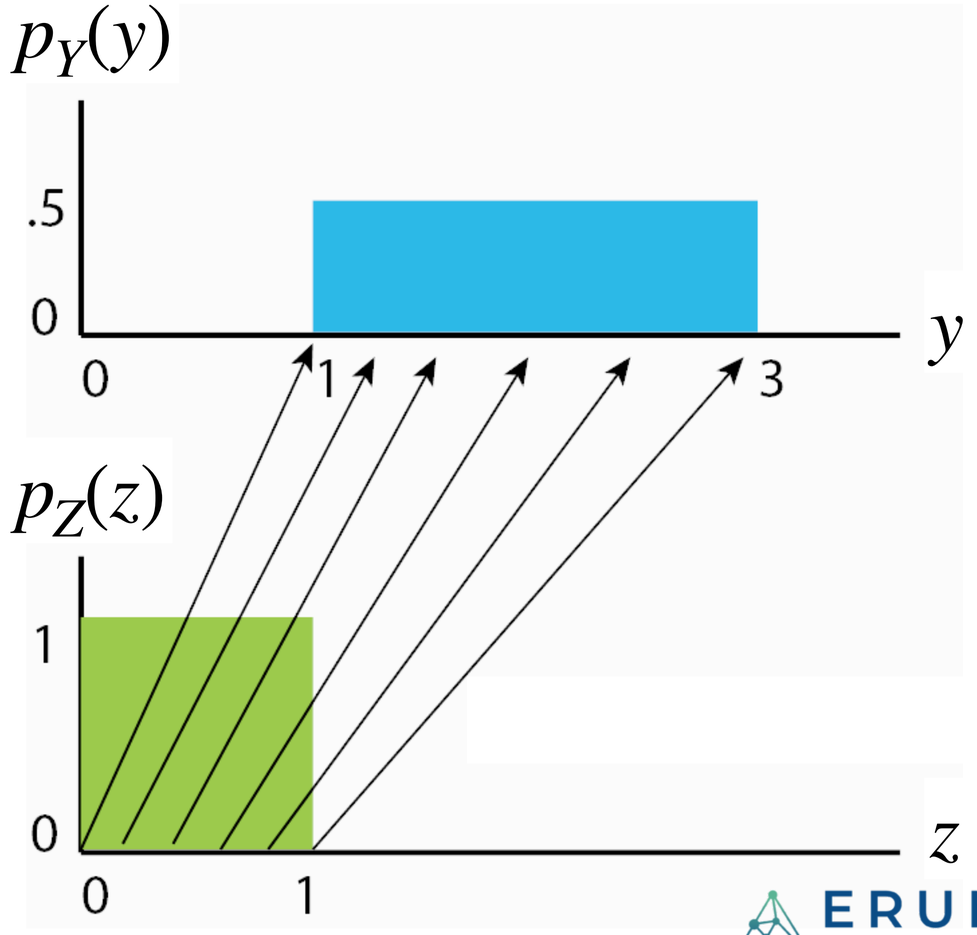
$$f(z) = 2z + 1 = y$$ **Affine transformation**

$f : \mathbb{R} \to \mathbb{R}$ is invertible ✓

$$z = f^{-1}(y) = (y - 1)/2, \qquad \frac{\partial f^{-1}(y)}{\partial y} = \frac{1}{2}$$

$$p_Y(y) = p_Z(f^{-1}(y)) \left| \frac{\partial f^{-1}(y)}{\partial y} \right|$$

$$\implies y \sim U[1,3] \qquad p_Y(y) = 0.5, \forall y \in [1,3]$$



https://blog.evjang.com/2018/01/nf1.html

Normalizing Flows

Volume morphing: it's all in the determinant of the jacobian

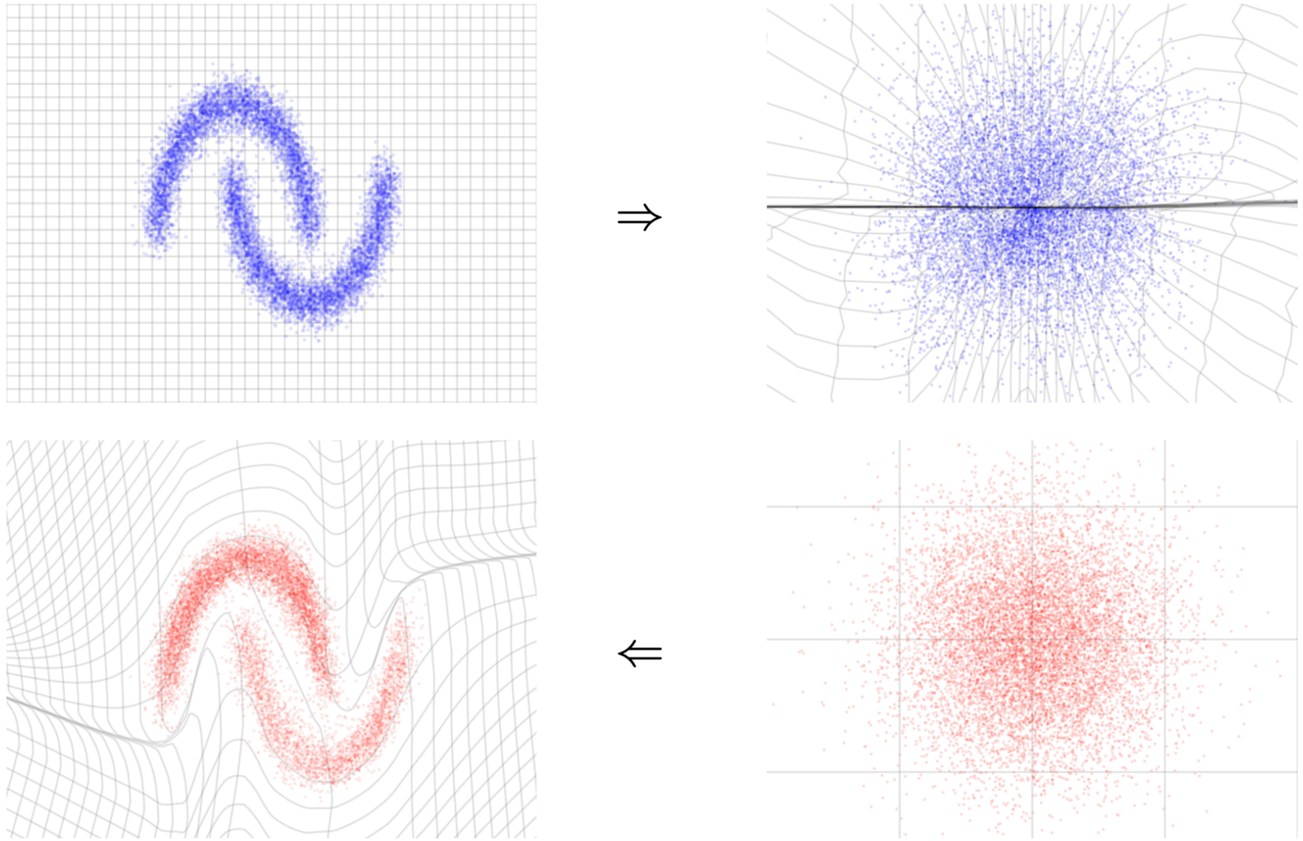
Inference

$$x \sim \hat{p}_X$$
$$z = f(x)$$

Generation

$$z \sim p_Z$$
$$x = f^{-1}(z)$$

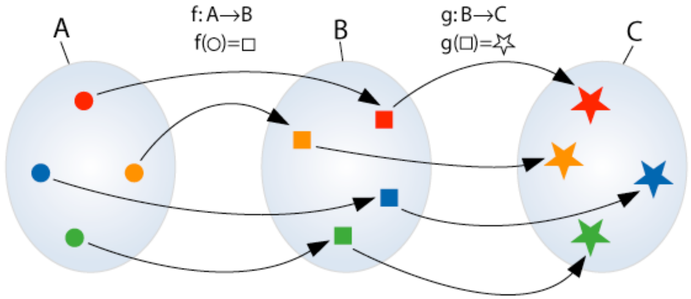
Data space $\mathcal{X}$ | Latent space $\mathcal{Z}$



https://arxiv.org/abs/1605.08803

ERUM DATA HUB

Normalizing Flows

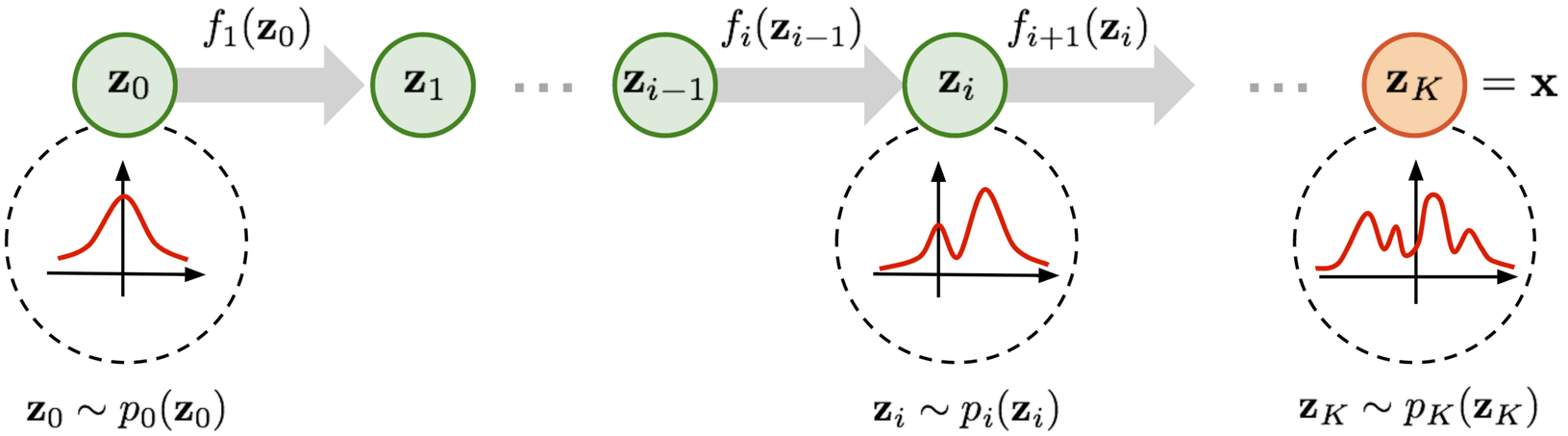


Central idea

The composition of invertible functions is **also** invertible

$$(f_1 \circ f_2)^{-1} = f_1^{-1} \circ f_2^{-1}$$

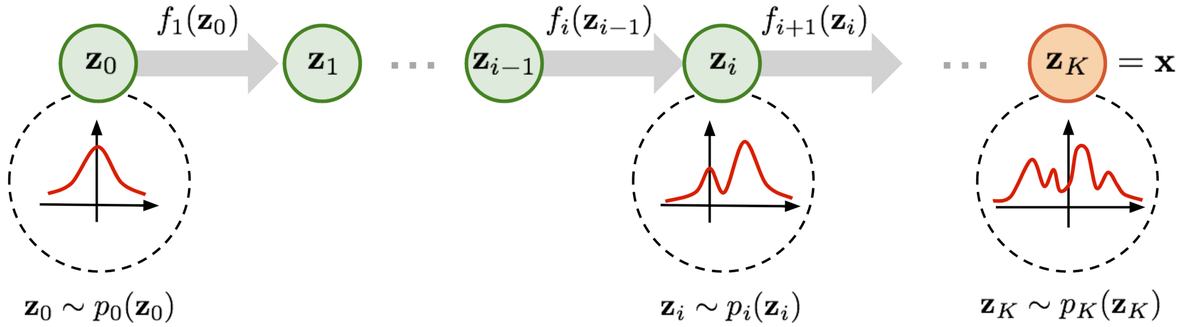
A Normalizing Flow attempts to transform the distribution $p_Z(z)$ gradually towards the more complex target distribution $p_X(x)$ by applying successive invertible transforms to it.



https://lilianweng.github.io/posts/2018-10-13-flow-models/

Normalizing Flows

How does the density transform?



$$\vec{z}_i \sim p_i(\vec{z}_i)$$

$$\vec{z}_i = f_i(\vec{z}_{i-1}) \implies \vec{z}_{i-1} = f_i^{-1}(\vec{z}_i) \implies p_i(\vec{z}_i) = p_{i-1}(f_i^{-1}(\vec{z}_i)) \left| \det \frac{\partial f_i^{-1}(\vec{z}_i)}{\partial \vec{z}_i} \right|$$

The *Theorem of the inverse function* tells us that if $y = f(x)$, then

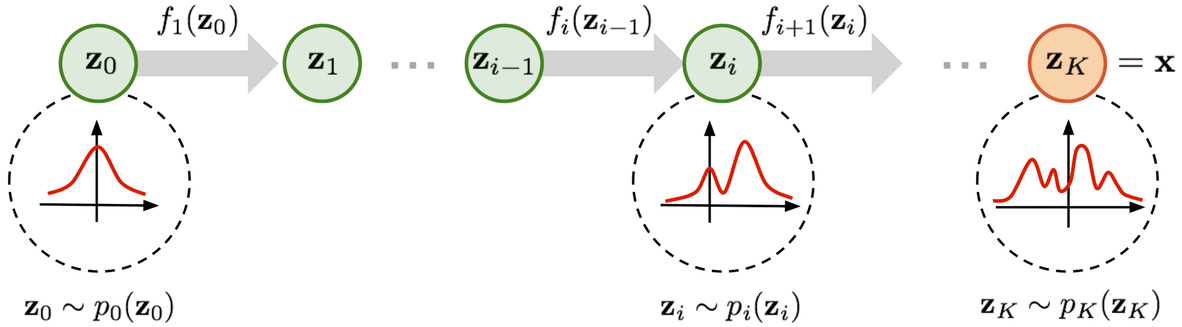
$$\frac{\partial f^{-1}(y)}{\partial y} = \left(\frac{\partial f(x)}{\partial x} \right)^{-1}$$

It's easy to see for 1D (valid for any dimension)

$$\frac{\partial f^{-1}(y)}{\partial y} = \frac{\partial x}{\partial y} = \left(\frac{\partial y}{\partial x} \right)^{-1} = \left(\frac{\partial f(x)}{\partial x} \right)^{-1}$$

Normalizing Flows

How does the density transform?



$$\implies p_i(\vec{z}_i) = p_{i-1}(f_i^{-1}(\vec{z}_i)) \left| \det \left(\frac{\partial f_i(\vec{z}_{i-1})}{\partial \vec{z}_{i-1}} \right)^{-1} \right|$$

$$\det(M)\det(M^{-1}) = \det(M \cdot M^{-1}) = \det(I) = 1 \implies \det(M^{-1}) = (\det(M))^{-1}$$

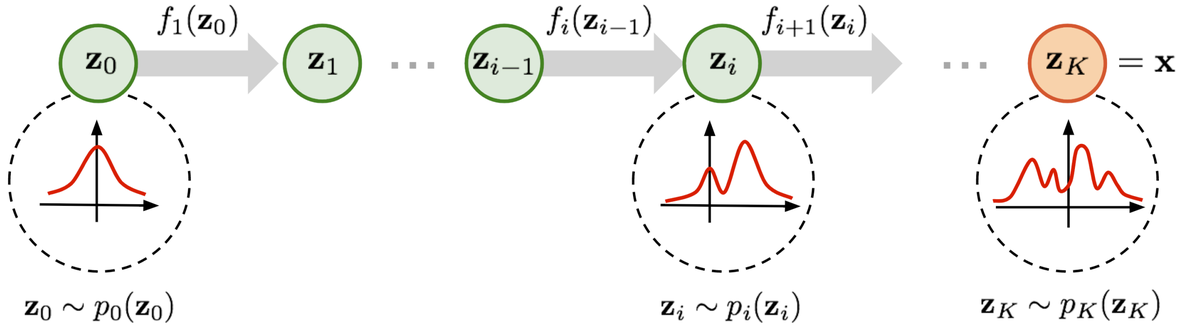
$$\implies p_i(\vec{z}_i) = p_{i-1}(f_i^{-1}(\vec{z}_i)) \left| \det \left(\frac{\partial f_i(\vec{z}_{i-1})}{\partial \vec{z}_{i-1}} \right) \right|^{-1}$$

Transform of the log-probability under an invertible function

$$\log p_i(\vec{z}_i) = \log p_{i-1}(\vec{z}_{i-1}) - \log \left| \det \frac{\partial f_i(\vec{z}_{i-1})}{\partial \vec{z}_{i-1}} \right|$$

Normalizing Flows

How does the density transform?



Transform of the log-probability under K invertible functions

$$\vec{x} = \vec{z}_K = f_K \circ f_{K-1} \circ \cdots \circ f_1(\vec{z}_0)$$

$$\log p_X(\vec{x}) = \log\left(p_{Z_K}(\vec{z}_K)\right) = \log\left(p_{Z_{K-1}}(\vec{z}_{K-1})\right) - \log\left|\det \frac{\partial f_K(\vec{z}_{K-1})}{\partial \vec{z}_{K-1}}\right|$$

$$\log p_X(\vec{x}) = \log\left(p_0(\vec{z}_0)\right) - \sum_{i=1}^{K} \log\left|\det \frac{\partial f_i(\vec{z}_{i-1})}{\partial \vec{z}_{i-1}}\right|$$

Sampling: $\vec{z}_0 \sim p_0 = \mathcal{N}_D(0, \mathbb{I})$ $\qquad \hat{x} = G(\vec{z}_0) = f_K \circ f_{K-1} \circ \cdots \circ f_1(\vec{z}_0)$

All nice and pretty… but where does the deep learning jump in?

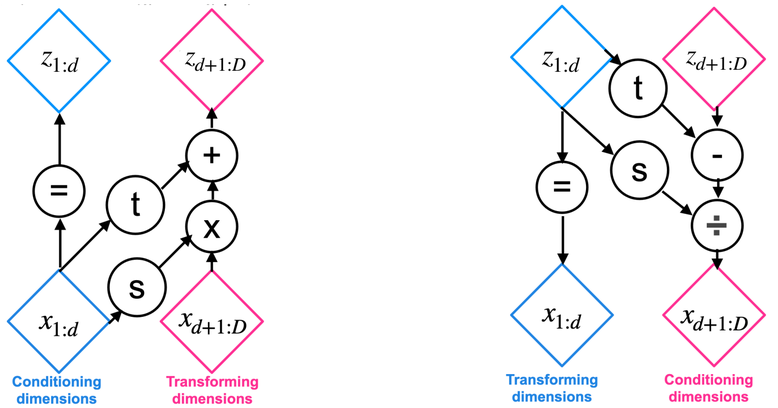
Normalizing Flows

Learning the transformations with Neural Networks (NNs)

We want transformations that are expressive enough, flexible, and also invertible.

We saw earlier that affine transformations were invertible!

$$z = ax + b$$



(a) Forward propagation (b) Inverse propagation

Affine Coupling Layer

Let $s_\theta(x)$ and $t_\theta(x)$ be Neural Networks parametrized by θ. We divide the input vector x in two parts: $\vec{x}_{1:d}$ and $\vec{x}_{(d+1):D}$.

Non-volume preserving (NVP)

$$\vec{z}_{1:d} = \vec{x}_{1:d}$$

$$\vec{z}_{(d+1):D} = \vec{x}_{(d+1):D} \odot \exp(s_\theta(\vec{x}_{1:d})) + t_\theta(\vec{x}_{1:d})$$

scaling *translation*

ERUM DATA HUB

Normalizing Flows

Affine Coupling Transform

Transform

$$\vec{z}_{1:d} = \vec{x}_{1:d}$$

scaling translation

$$\vec{z}_{(d+1):D} = \vec{x}_{(d+1):D} \odot \exp(s_\theta(\vec{x}_{1:d})) + t_\theta(\vec{x}_{1:d})$$

The invertibility is a consequence of the fact that we divide the vector in two and we leave half untouched. In this way, we store some sort of **memory** of the input vector in the output of the operation.

Inverse

$$\vec{x}_{1:d} = \vec{z}_{1:d}$$

$$\vec{x}_{(d+1):D} = (\vec{z}_{(d+1):D} - t(\overrightarrow{y_{1:d}})) \odot \exp(-s(\vec{y}_{1:d}))$$

Jacobian

$$\mathbf{J} = \begin{bmatrix} \mathbb{I}_d & 0 \\ \frac{\partial \vec{z}_{(d+1):D}}{\partial \vec{x}_{1:d}} & \text{diag}(\exp(s_\theta(\vec{x}_{1:d}))) \end{bmatrix}$$

Lower triangular

ERUM DATA HUB

Normalizing Flows

Affine Coupling Transform

$$\mathbf{J} = \begin{bmatrix} \mathbb{I}_d & 0 \\ \frac{\partial \vec{y}_{(d+1):D}}{\partial \vec{x}_{1:d}} & \text{diag}(\exp(s_\theta(\vec{x}_{1:d}))) \end{bmatrix}$$

Remember that the determinant of a triangular matrix is the product of its diagonal elements!

$$\det \mathbf{J} = \exp(\sum_j s_\theta(\vec{x}_{1:d}))$$

$$\log p_X(\vec{x}) = \log\left(p_0(\vec{z}_0)\right) - \sum_{i=1}^{K} \log \left| \det \frac{\partial f_i(\vec{z}_{i-1})}{\partial \vec{z}_{i-1}} \right| \implies \boxed{\log p_X(x) = \log p_0(\vec{z}_0) - \sum_{j=1}^{D-d} s_\theta(\vec{x}_{1:d})}$$

By controlling the structure of the transformation, we have achieved an invertible transformation that is flexible and expressive enough (thanks to the NNs).

Notebook Normalizing Flows - Part 1

ERUM DATA HUB

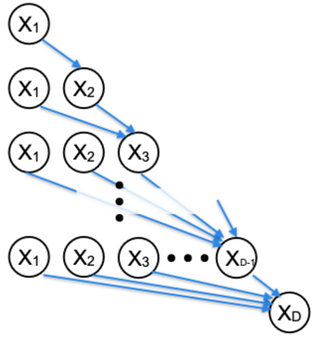
Normalizing Flows

Masked Autoregressive Flow (MAF)

Another possibility would be to model the density of vector $\vec{x}$ as the product of 1D conditional densities

$$p(\vec{x}) = \prod_{i=1}^{D} p(x_i \,|\, x_{1:i-1})$$

$$p(x_i \,|\, x_{1:i-1}) = \mathcal{N}(x_i \,|\, m_{i,\theta}(x_{1:i-1}), (\exp s_{i,\theta}(x_{1:i-1}))^2)$$



Each dimension is standardized using statistics that are conditioned to the previous dimensions.

$$z_i = (x_i - m_{i,\theta}(x_{1:i-1}))/\exp s_{i,\theta}(x_{1:i-1})$$ *Autoregressive standardization*

Inverse

$$x_i = z_i \exp s_{i,\theta}(x_{1:i-1}) + m_{i,\theta}(x_{1:i-1}),\ z_i \sim \mathcal{N}(0,1)$$

Normalizing Flows

Masked Autoregressive Flow (MAF)

How would the Jacobian be under this transformation?

$$z_i = (x_i - m_{i,\theta}(x_{1:i-1}))/\exp s_{i,\theta}(x_{1:i-1}) \qquad \frac{dz_i}{dx_j}?$$

It defines an invertible transform with a triangular Jacobian!

$$\log|\det J| = -\sum_i s_{i,\theta}(x_{<i})$$

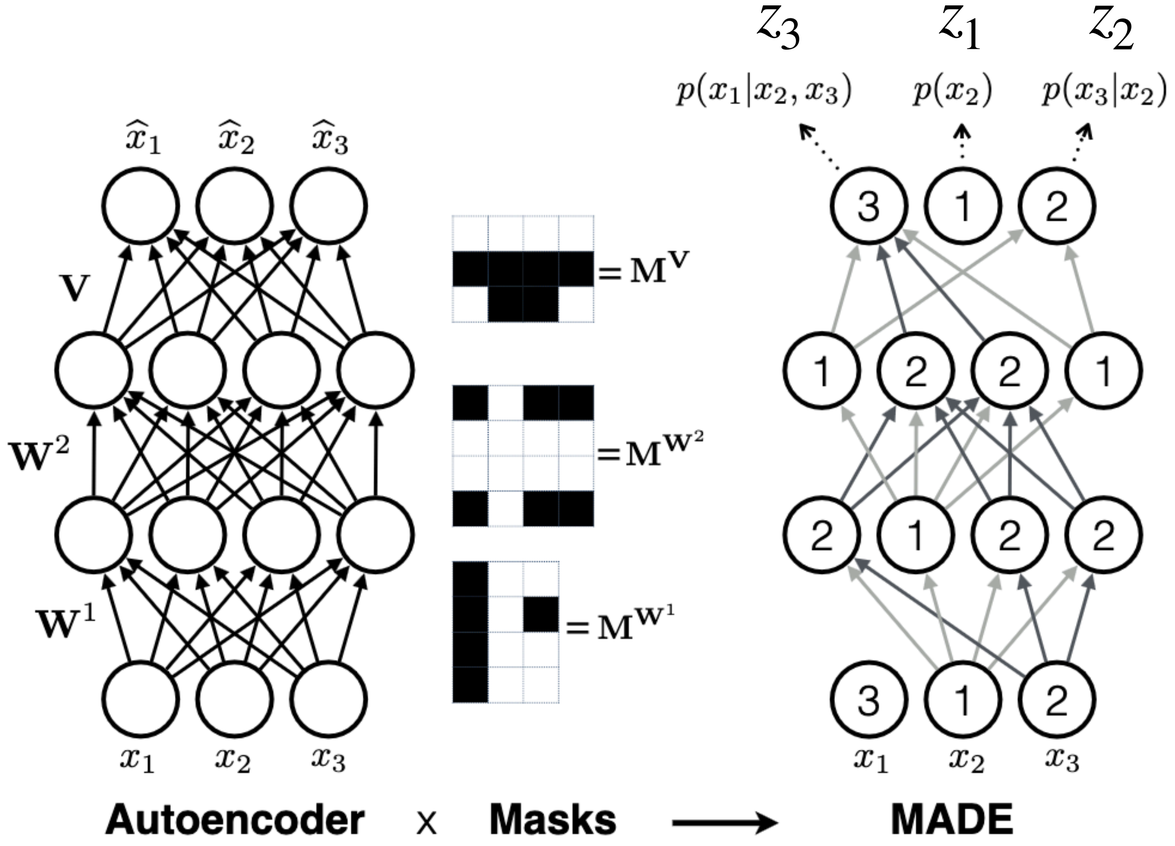
$$x_{1:i-1} = x_{<i}$$

$$\log p_X(x) = \log p_0(\vec{z}_0) + \sum_{i=1}^{D} s_{i,\theta}(\vec{x}_{<i})$$

How can we build m_θ and s_θ so that they fulfill the conditional dependency?

Normalizing Flows

Masked Autoencoder for Distribution Estimation (MADE)



Masks force each output i to depend on $x_{<i}$

MAF combines an autoregressive network (MADE) with the change of variables to construct an invertible generative model

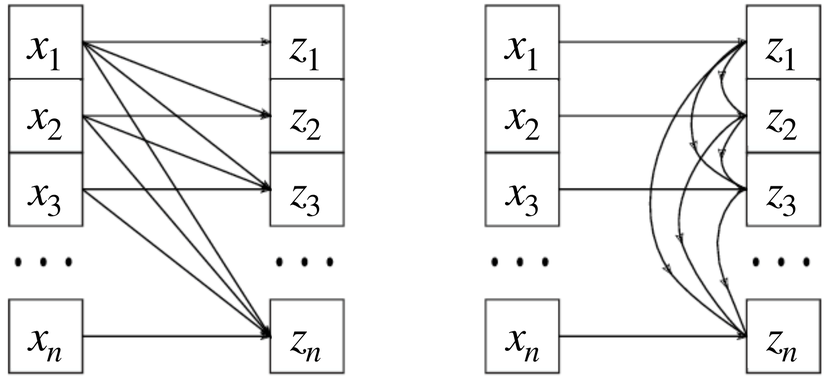
Super parallelizable!

$$z_i = (x_i - m_{i,\theta}(x_{1:i-1}))/\exp s_{i,\theta}(x_{1:i-1})$$

https://arxiv.org/abs/1502.03509

Normalizing Flows

Inverse Autoregressive Flow (IAF)



MAF

With MADE

$$z_i = (x_i - m_{i,\theta}(x_{1:i-1}))/\exp s_{i,\theta}(x_{1:i-1})$$ ←*We have everything, parallelizable, FAST!*

Generation

$$\hat{x}_i = z_i \exp s_{i,\theta}(\hat{x}_{1:i-1}) + m_{i,\theta}(\hat{x}_{1:i-1}),\ z_i \sim \mathcal{N}(0,1)$$ ←*We need to calculate $\hat{x}_{1:i-1}$ for each $\hat{x}_i$*

SLOW

IAF

$$z_i = (x_i - m_{i,\theta}(z_{1:i-1}))/\exp s_{i,\theta}(z_{1:i-1})$$ ←*SLOW*

Generation

$$\hat{x}_i = z_i \exp s_{i,\theta}(z_{1:i-1}) + m_{i,\theta}(z_{1:i-1}),\ z_i \sim \mathcal{N}(0,1)$$ ←*FAST*

$$\log|\det J| = \sum_i s_{i,\theta}(z_{<i})$$

Normalizing Flows

Neural Spline Flows (NSF)

Why use a simple affine transform? Maybe we can increase expressivity with a bit of complexity

$$y = g_\theta(x)$$

Quadratic $g_{\theta,k}(x) = a_k x^2 + b_k x + c_k$

https://arxiv.org/abs/1906.04032

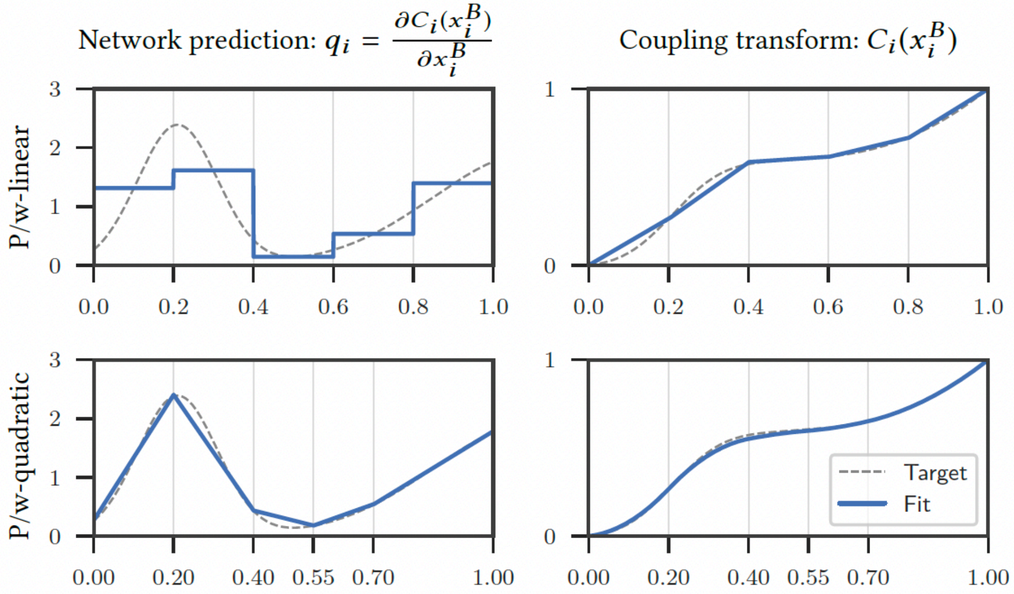
Cubic $g_{\theta,k}(x) = a_k x^2 + b_k x + c_k + d_k x^3$

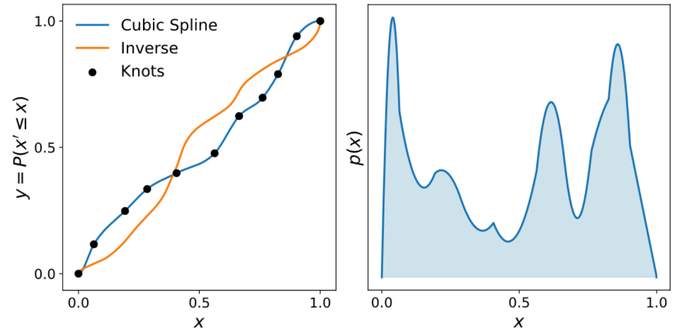
https://arxiv.org/abs/1906.02145

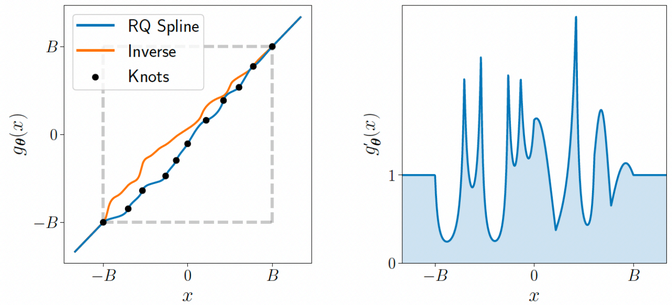
Rational Quadratic $g_{\theta,k}(x) = \dfrac{a_k x^2 + b_k + c_k}{d_k x^2 + e_k + f_k}$

https://arxiv.org/abs/1906.04032

- We need to be able to evaluate de Jacobian
- It needs to be monotonically increasing
- Continuous derivative







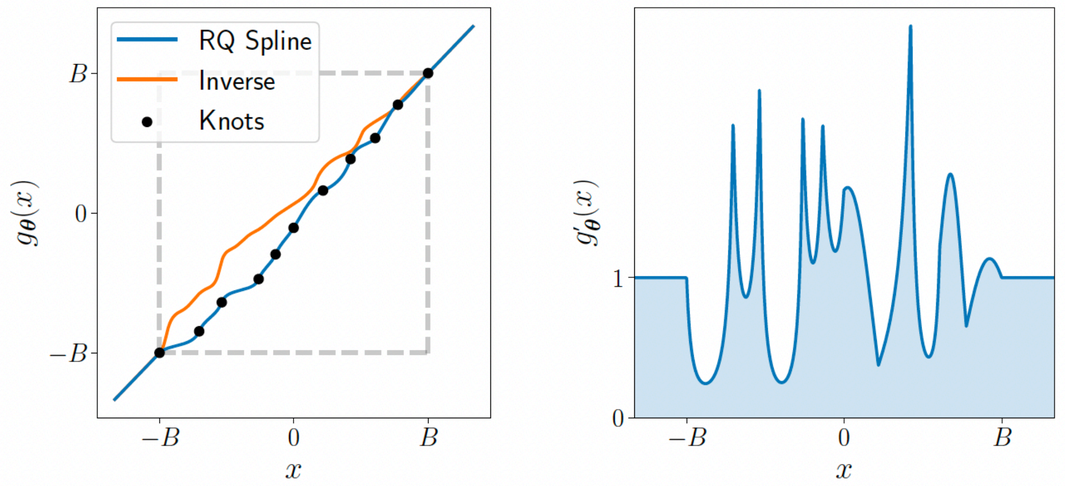
Can vary nonlinear inside each bin!

ERUM DATA HUB

Normalizing Flows

Neural Spline Flows (NSF)

$$g_{\theta,k}(x) = \frac{a_k x^2 + b_k + c_k}{d_k x^2 + e_k + f_k}$$



Steps:

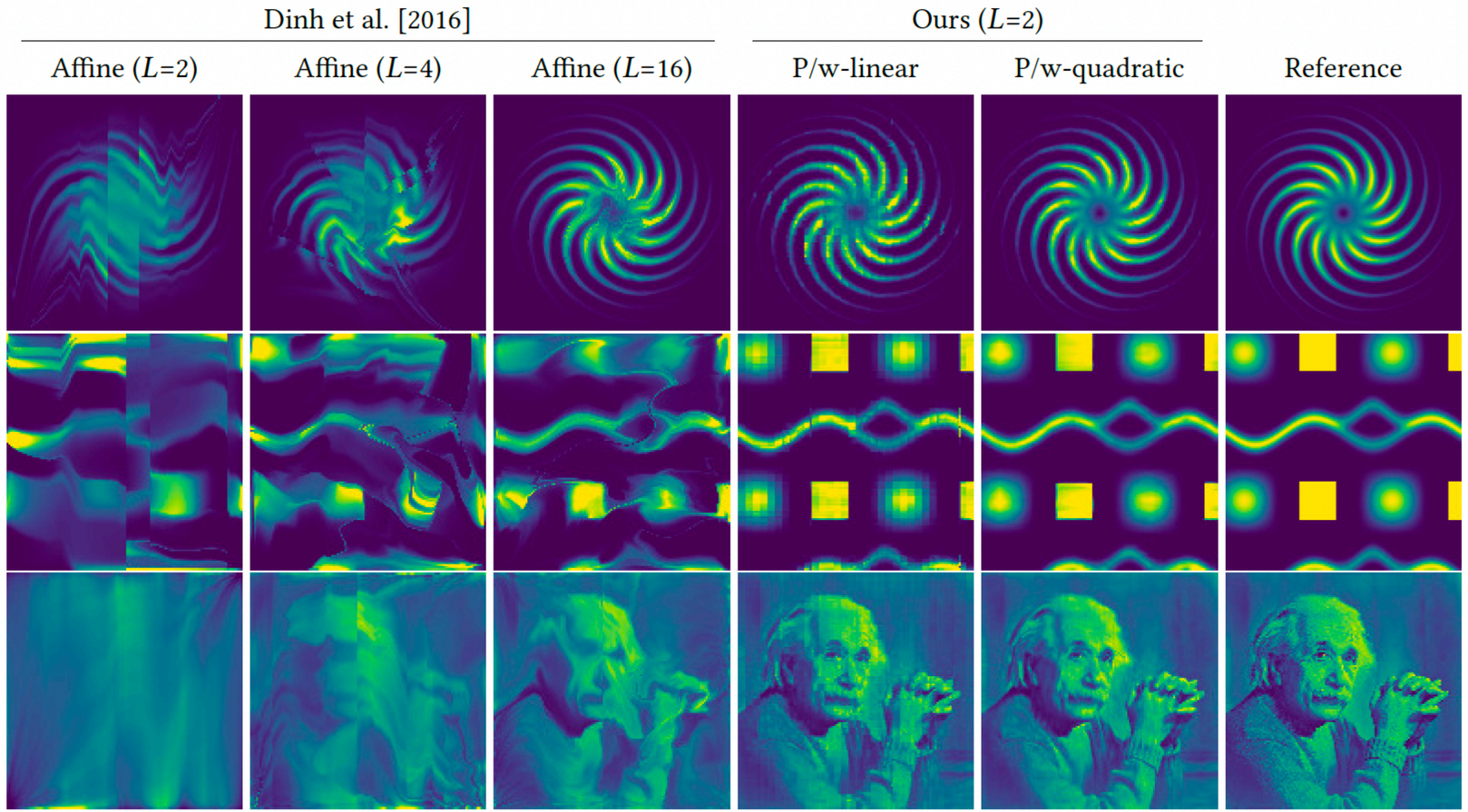
1. A NN takes input $x_{1:d}$ and outputs a $\theta_i \in \mathbb{R}^{3K-1}$ for each $i = d+1, D$
2. We divide $\theta_i = [\theta_i^w, \theta_i^h, \theta_i^d]$, with $\theta_i^w, \theta_i^h \in \mathbb{R}^K$ and $\theta_i \in \mathbb{R}^{K-1}$

$$p_Y(\vec{y}) = p_X(g_\theta^{-1}(\vec{y})) \left| \det \frac{\partial g_\theta(\vec{y})}{\partial \vec{y}} \right|^{-1}$$

3. Use a Softmax to scale θ_i^w and θ_i^h increment between $(-B, B)$ and (B, B) to get the Δx_k and Δy_k of the spline bins.
4. Pass θ_i^d through a positive function to get the derivatives of the internal knots.

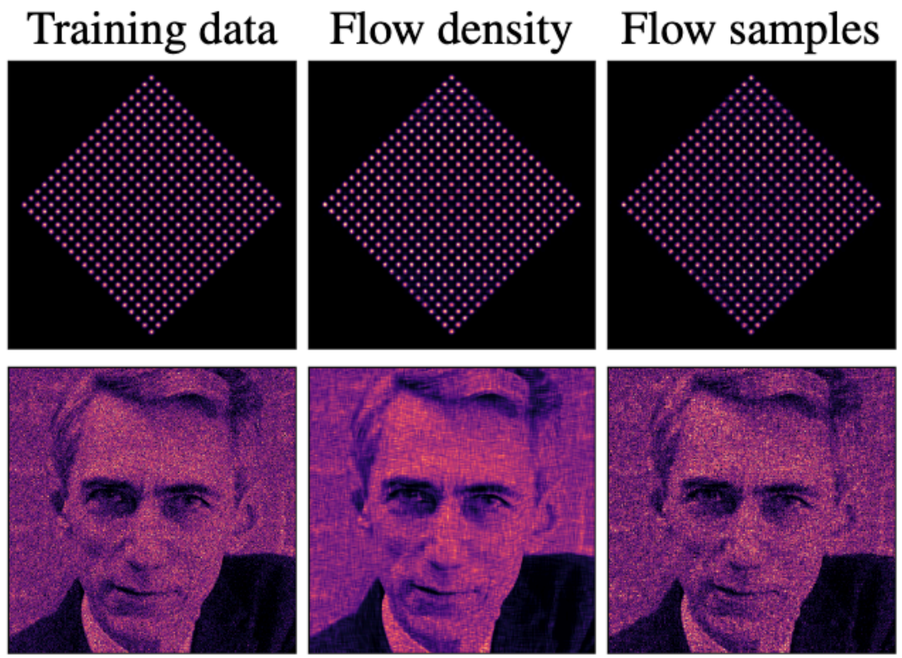
Normalizing Flows

Neural Spline Flows (NSF)



https://arxiv.org/abs/1906.04032

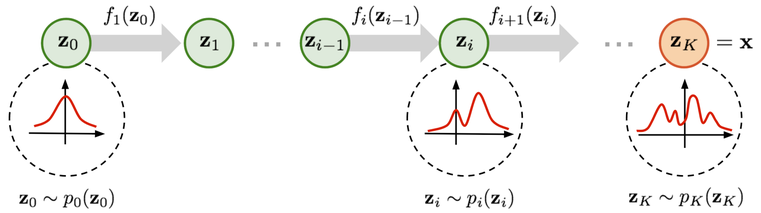
Rational quadratic



https://arxiv.org/abs/1906.04032

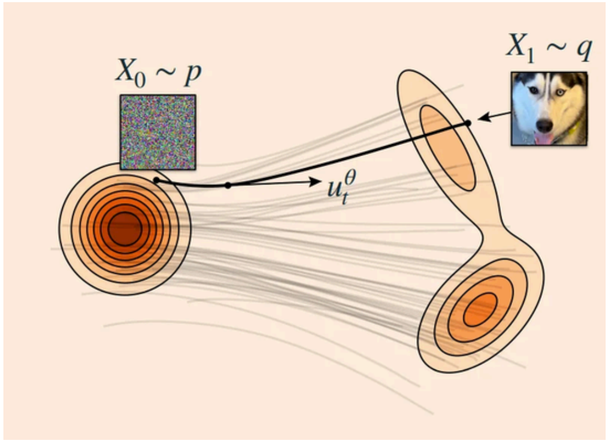
Notebook Normalizing Flows - Part 2

Continuous Normalizing Flows



Overcoming limitations

- NFs architectures have to be invertible
- We must define a finite number of transformations or layers



Can we think on a continuous transformation?

Let's introduce an artificial time t such that at $t = 0$ we are in our a priori distribution and at time $t = 1$ we reach the target distribution

Neural ODE

$$\frac{\partial x(t)}{\partial t} = f_\theta(x(t), t)$$

Ordinary Differential Equation (ODE)

Now we only have one network f_θ, which we evaluate at different points in the time interval $t \in [0,1]$

ERUM DATA HUB

Continuous Normalizing Flows

Jacobian

Refresher: Discrete case

$$\vec{x} = \vec{z}_K = f_K \circ f_{K-1} \circ \cdots \circ f_1(\vec{z}_0)$$

Measures the finite global volume change

$$\log p_X(\vec{x}) = \log\left(p_{Z_K}(\vec{z}_K)\right) = \log\left(p_{Z_{K-1}}(\vec{z}_{K-1})\right) - \log\left|\det \frac{\partial f_K(\vec{z}_{K-1})}{\partial \vec{z}_{K-1}}\right|$$

Continuous case

$$\log p_X(\vec{x}) = \log\left(p_0(\vec{z}_0)\right) - \sum_{i=1}^{K} \log\left|\det \frac{\partial f_i(\vec{z}_{i-1})}{\partial \vec{z}_{i-1}}\right|$$

$$\frac{\partial x(t)}{\partial t} = f_\theta(x(t), t)$$

(Using continuity equation and conservation of probability)

Adjoint method for Back Propagation

$$\frac{d}{dt}\log p(x(t)) = -\,Tr\left(\frac{\partial f_\theta(x(t), t)}{\partial x}\right) \implies \log p(x(T)) = \log p(x(0)) - \int_0^T Tr\left(\frac{\partial f_\theta(x(t), t)}{\partial x(t)}\right) dt$$

divergence

To obtain the transformation, we need to solve an ODE numerically.

Continuous Normalizing Flows

Jacobian

Several infinitesimal transformations $x(t + dt) = x(t) + f(x(t), t)dt$

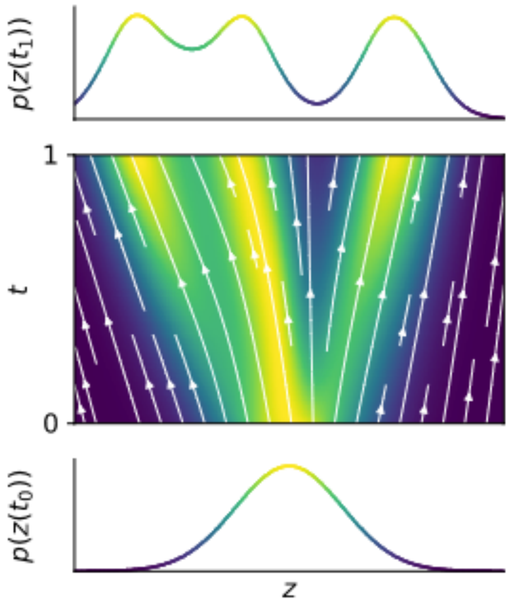
$$J = I + \frac{\partial f}{\partial x} dt$$

Using that $\det(I + Adt) \approx 1 + Tr(A)dt$ (first order Taylor expansion)

$$\log \det(J) \approx Tr\left(\frac{\partial f}{\partial x}\right) dt$$

$$\frac{d}{dt} \log p(x(t)) = - Tr\left(\frac{\partial f_\theta(x(t), t)}{\partial x}\right) \implies \log p(x(T)) = \log p(x(0)) - \int_0^T Tr\left(\frac{\partial f_\theta(x(t), t)}{\partial x(t)}\right) dt$$

Normalizing Flows



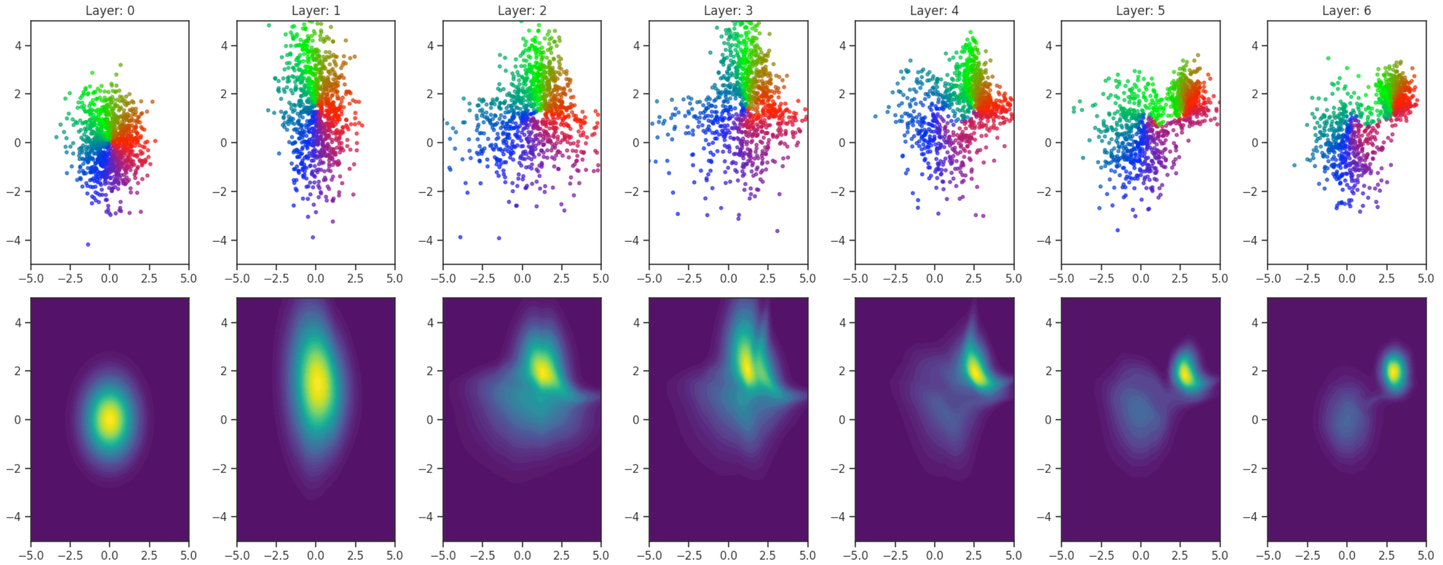
Invertibility comes for free

$$x(0) \rightarrow x(T)$$

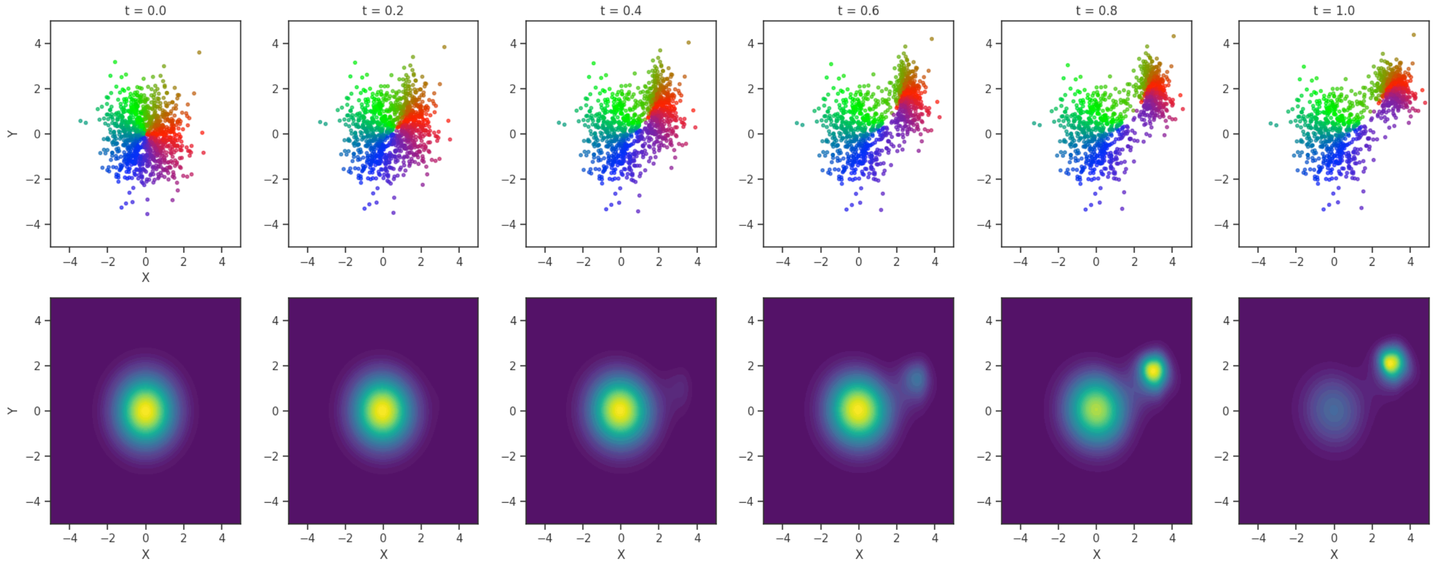
$$x(T) \rightarrow x(0)$$

https://arxiv.org/pdf/1810.01367

Discrete



Continuous

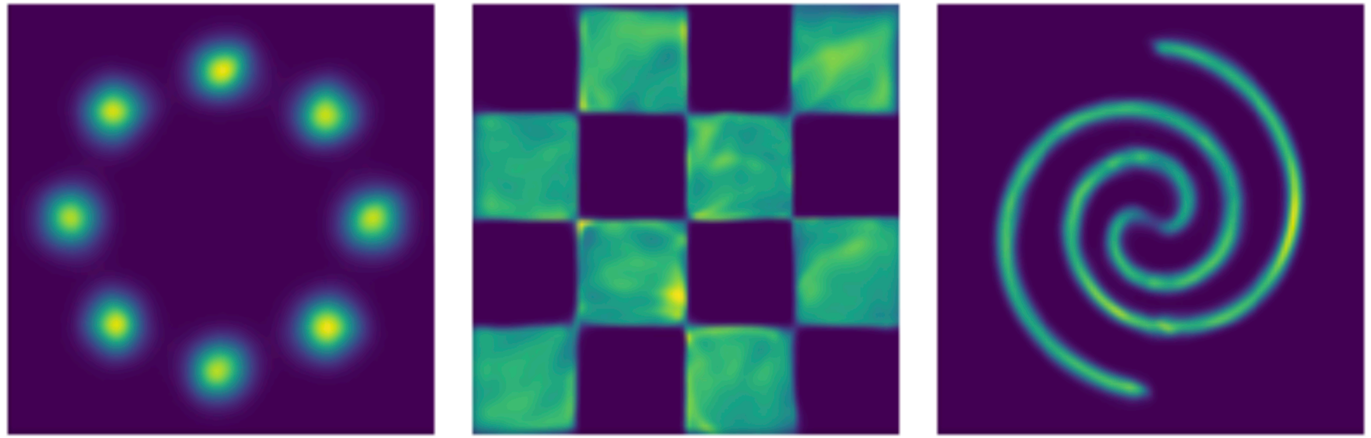


Normalizing Flows

Continuous Normalizing Flows

Target

neural ODE



Notebook Normalizing Flows - Part 3

Extra: Dequantization

Discrete distributions (images, Glow)

- We have seen NFs on continuous distributions
- Many data have discrete distributions.
- We can convert discrete data in continuous data by adding a bit of noise

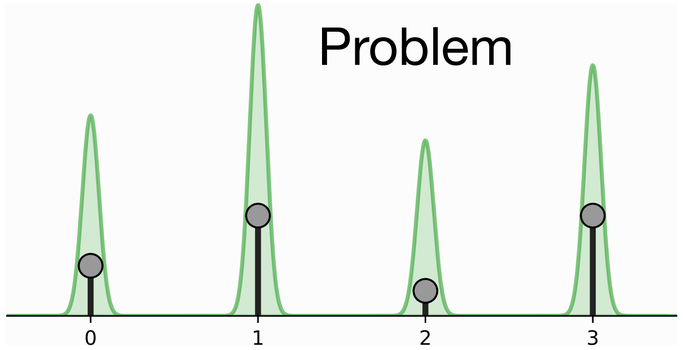
1. $v = x + u,\ u \sim U[0,1]$, where x are the original discrete data

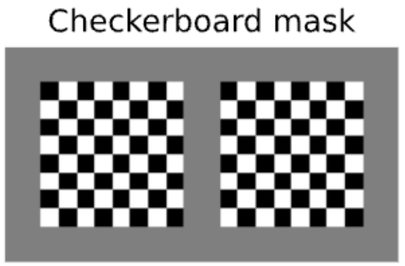
$$p(x) = \int p(x+u)du = \int \frac{q(u\,|\,x)}{q(u\,|\,x)} p(x+u)du = E_{u\sim q(u|x)} \frac{p(x+u)}{q(u\,|\,x)} = E_{u\sim U[0,1]^d} p(x+u)$$

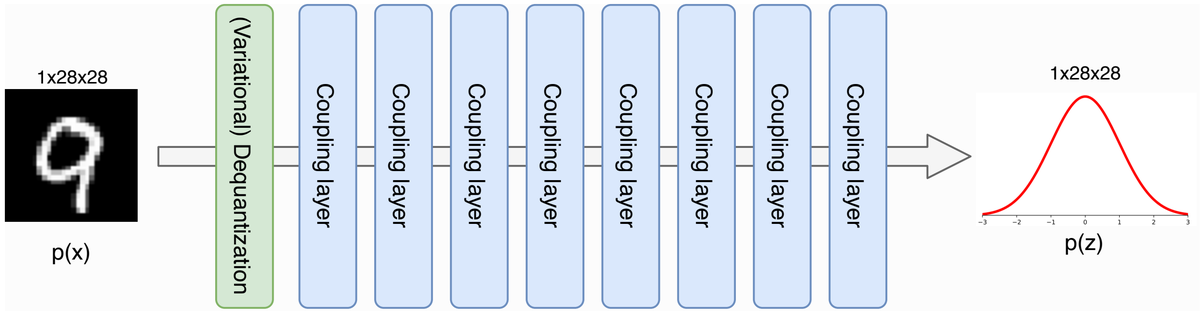
2. Variational Dequantization: we use a learnable distribution $q_\theta(u\,|\,x)$, modeled by an additional normalizing flow!

Extra: Dequantization

Example



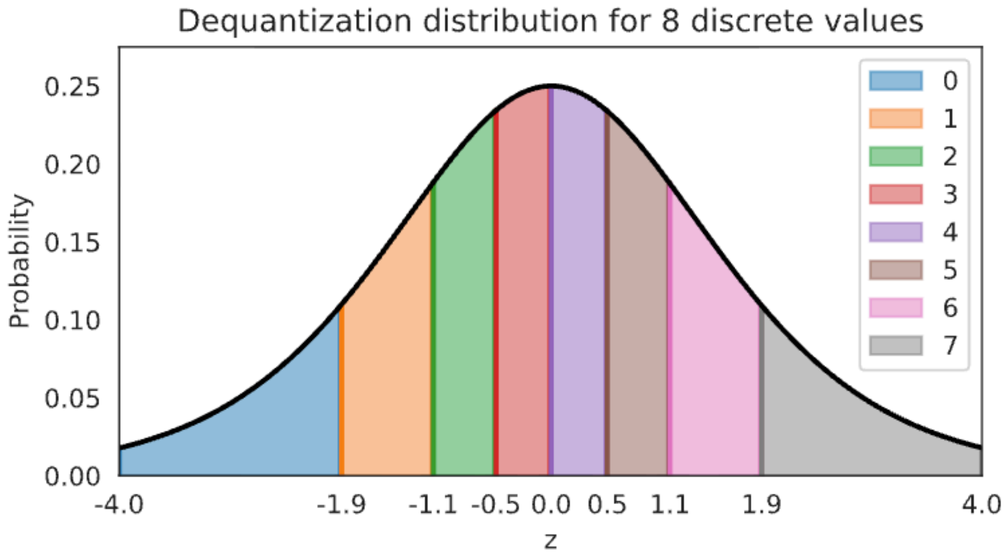


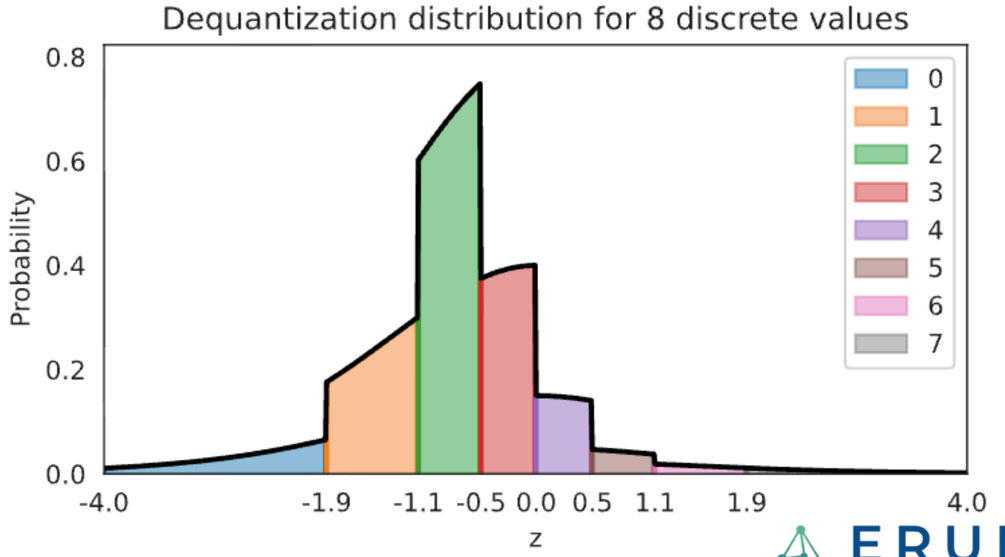


$$z = (x + u)/256,\ u \sim U[0,1] \implies z \in [0,1]$$

$$logit(z) = \log(z/(z-1)) \in \mathbb{R}$$

Resulting Logistic distribution



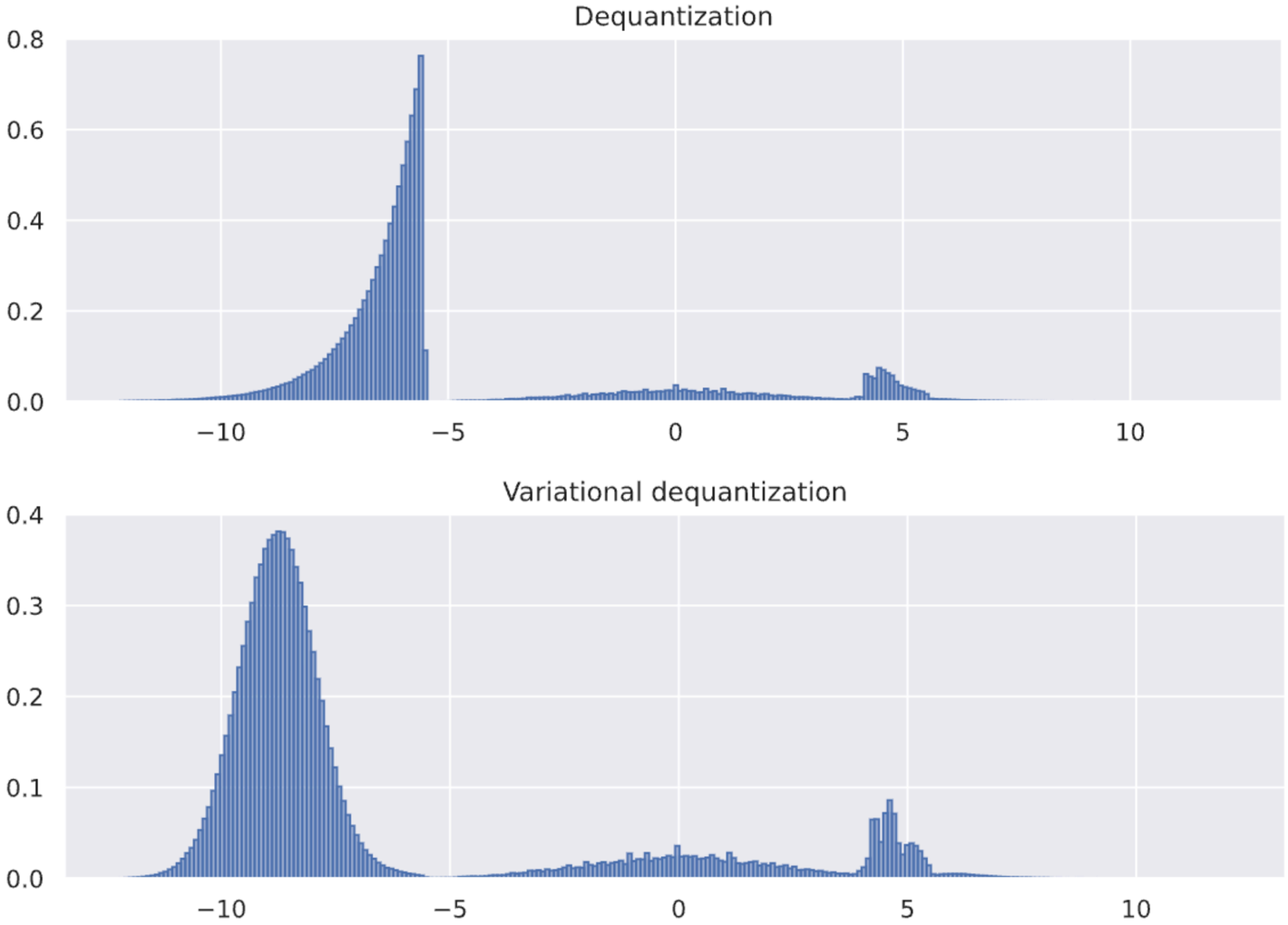


What happens if it is not uniform

ERUM DATA HUB

Extra: Dequantization

Example



ERUM DATA HUB

DEEP LEARNING SCHOOL

"ADVANCED CONCEPTS"

4TH EDITION

Thank you for your attention!
Let it Flow

Instituto Balseiro

CONICET

JÜLICH Forschungszentrum

ERUM DATA HUB